Reinforcement Learning for Market Making: Surpassing the Avellaneda-Stoikov Benchmark with PPO and SAC

Avinash Chauhan
School Of Mathematics
INDIAN INSTITUTE OF SCIENCE EDUCCATION AND RESEARCH THIRUVANANTHAPURAM
Email: avinash23@iisertvm.ac.in

***Abstract*—Market making is a sequential decision problem in which a liquidity provider must quote bid and ask prices while balancing spread capture against inventory risk and adverse selection. The classical Avellaneda-Stoikov model provides an elegant analytical solution under simplifying assumptions, but real or simulated market dynamics often violate those assumptions. This paper studies whether reinforcement learning can learn a more adaptive quoting policy than the Avellaneda-Stoikov benchmark. The main focus is on two continuous-control algorithms, Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC), because their action spaces match the continuous quoting objective naturally. Additional methods such as DQN, QR-DQN, A2C, DDPG, TD3, and TRPO are included as comparison baselines. We present the mathematical formulation of market making, derive the classical benchmark, explain PPO and SAC in detail, describe the simulation environments, and compare the resulting policies using the figures generated from the implemented notebooks.**

***Index Terms*—Reinforcement Learning, Market Making, PPO, SAC, Avellaneda-Stoikov, Algorithmic Trading, Continuous Control**

I. Introduction

Market making is one of those beautiful little financial tasks where the agent is paid to stand in the middle of uncertainty and hope the spread is enough to compensate for reality. A market maker posts bid and ask quotes, executes against incoming orders, and attempts to earn the spread while avoiding harmful inventory accumulation. The problem is not static. Prices move, order flow changes, and inventory that looks harmless in one regime can become a disaster in another.

The classical Avellaneda-Stoikov framework gives a closed-form quoting rule under idealized assumptions. It is useful, elegant, and mathematically satisfying. Unfortunately, markets rarely agree to be elegant. This project therefore asks a practical question: can an RL agent, trained in simulation, learn a quoting policy that is more adaptive than the Avellaneda-Stoikov solution in the chosen environments? To answer this, PPO and SAC are treated as the main methods because they operate directly in continuous action spaces and are therefore aligned with the market making objective.

II. Problem Formulation

We model market making as a Markov Decision Process (MDP) defined by $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. At each time step t, the agent observes a state s_t, chooses an action a_t, receives reward r_t, and transitions to s_{t+1}.

A useful state representation is

$$s_t = (p_t, q_t, \Delta p_t, \sigma_t, \tau_t, x_t), \tag{1}$$

where p_t is the mid-price, q_t is inventory, Δp_t is recent price change, σ_t is volatility, τ_t is remaining time, and x_t includes auxiliary features such as spread, wealth, or toxicity.

The action controls quote placement:

$$a_t = (\delta_t^{\text{bid}}, \delta_t^{\text{ask}}), \tag{2}$$

where the offsets determine how far the bid and ask are placed from the mid-price. A sensible reward should encourage executed spread while discouraging extreme inventory:

$$r_t = \Delta\text{PnL}_t - \lambda_q q_t^2 - \lambda_c C_t. \tag{3}$$

Here ΔPnL_t is incremental profit and loss, q_t^2 penalizes inventory buildup, and C_t represents execution cost or adverse selection. The coefficient λ_q controls how aggressively the agent is punished for carrying inventory.

The learning objective is to maximize expected discounted return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \tag{4}$$

The entire project is built around this objective, with the additional requirement that the learned policy should be compared against the analytical Avellaneda-Stoikov benchmark.

III. Classical Benchmark: Avellaneda-Stoikov

The Avellaneda-Stoikov model assumes that the market maker has exponential utility

$$U(X_T) = -e^{-\gamma X_T}, \tag{5}$$

where X_T is terminal wealth and γ is the risk-aversion parameter. Inventory evolves through order execution, and the objective is to maximize expected utility of terminal wealth. Under the usual assumptions of independent order arrivals and diffusive mid-price evolution, dynamic programming leads to a reservation price

$$r_t = p_t - q_t \gamma \sigma^2 (T - t), \tag{6}$$

and a half-spread of the form

$$\delta_t^* = \frac{\gamma\sigma^2(T-t)}{2} + \frac{1}{\gamma}\ln\left(1+\frac{\gamma}{k}\right). \tag{7}$$

The bid and ask quotes are then

$$p_t^b = r_t - \delta_t^*, \qquad p_t^a = r_t + \delta_t^*. \tag{8}$$

This benchmark is elegant because it directly links spread choice to inventory and volatility. It is also limited because the derivation relies on assumptions that often fail in richer environments. That is precisely why a learned policy is interesting: it can adapt to situations where the closed-form model is too rigid.

IV. Market Simulation Environments

A. Continuous Market Environment

The main training environment evolves continuously and is designed to match the assumptions of continuous-control algorithms. The mid-price follows a stochastic process

$$dp_t = \mu\, dt + \sigma\, dW_t, \tag{9}$$

where μ is drift, σ is volatility, and W_t is Brownian motion. Order execution depends on the quote offset and market pressure.

B. Drift-and-Jump Stress Environment

To test robustness, a second environment includes sudden price jumps:

$$dp_t = \mu\, dt + \sigma\, dW_t + J_t dN_t, \tag{10}$$

where N_t is a Poisson process and J_t is the jump size. This setting is useful because a policy that is only good in smooth markets is not really useful in markets that jump like they have a personal grudge.

C. Toxic Flow and LOB Replayer

A third environment exposes a toxicity feature ξ_t, which acts as a signal of adverse order flow. Fill probability can be written as

$$\mathbb{P}(\text{fill}_t = 1) = f(\delta_t, \xi_t). \tag{11}$$

As toxicity rises, the policy should reduce exposure or widen spreads.

V. Proximal Policy Optimization (Main Method I)

PPO is one of the main methods in this project because it naturally handles continuous quote offsets. In market making, the agent must output bid and ask distances that are not merely discrete labels but actual continuous decisions. PPO is appropriate because it learns a stochastic policy and updates it carefully, avoiding abrupt changes that could destabilize training. The policy is parameterized as $\pi_\theta(a|s)$, and the central idea is to improve it using the policy-gradient theorem while restricting the update size.

The basic policy-gradient objective can be written as

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi\left[\nabla_\theta \log \pi_\theta(a_t|s_t) A_t\right], \tag{12}$$

where A_t is the advantage estimate. For market making, advantage measures whether the chosen quote improved value relative to the local baseline. A positive advantage means the chosen spread and inventory response were better than expected; a negative value means the agent probably overquoted, underquoted, or otherwise behaved like a cautious spreadsheet with delusions of liquidity provision.

The major instability in naive policy gradients is that the policy can change too much in a single update. PPO fixes this using a clipped surrogate objective. Let

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}. \tag{13}$$

The clipped objective is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right]. \tag{14}$$

This means the new policy cannot move too far from the old one when the update would be too aggressive. In market making, this matters because quote instability can produce oscillating spreads and unpredictable inventory swings.

The critic estimates the value function $V_\phi(s)$ using mean-squared error against discounted returns:

$$\mathcal{L}_V(\phi) = \mathbb{E}\left[(V_\phi(s_t) - \hat{R}_t)^2\right]. \tag{15}$$

The advantage can be formed using temporal-difference residuals or generalized advantage estimation. In this project, PPO is used as the primary continuous-control method because it is stable, easy to tune relative to more delicate actor-critic methods, and well matched to the task of quoting bid and ask prices. It is also a strong candidate for outperforming the Avellaneda-Stoikov benchmark because it can adapt to nonlinear reward structure, asymmetric market regimes, and toxicity-driven behavior that is not captured by the closed-form model.

Algorithm 1 PPO Training

```
Initialize actor π_θ and critic V_φ
Collect on-policy rollouts using the current policy
Compute returns and advantages
Compute probability ratios r_t(θ)
Optimize the clipped objective for several epochs
Update critic using value regression
Replace the old policy with the updated policy
```

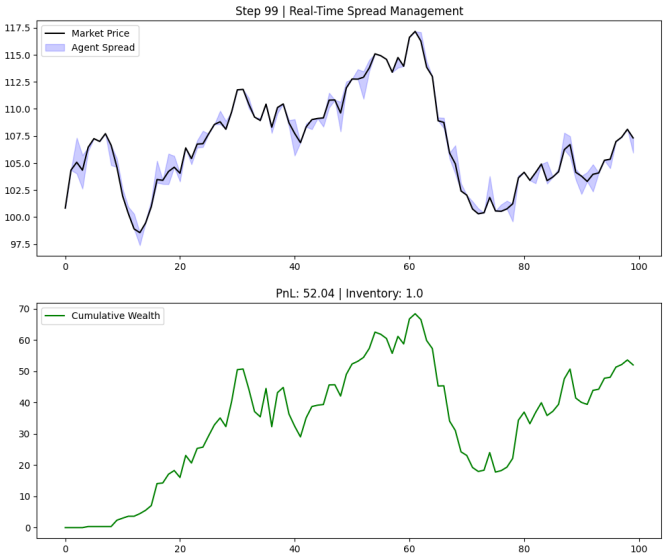


Fig. 1: PPO simulation output in the market dynamics environment.

VI. Soft Actor-Critic (Main Method II)

SAC is the second main method because it also supports continuous actions but introduces entropy maximization, which is extremely useful in noisy market making environments. The key intuition is that a good market maker should not collapse too early into a deterministic quoting pattern. If the market changes regime or the reward landscape becomes multi-modal, an entropy-aware policy can continue exploring better quoting strategies instead of locking itself into a brittle local solution.

SAC optimizes the maximum-entropy objective

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\pi}\left[r_t + \alpha\mathcal{H}(\pi(\cdot|s_t))\right], \tag{16}$$

where α is the temperature parameter and $\mathcal{H}$ is policy entropy. The entropy term rewards policies that retain some uncertainty, which in market making often translates to healthier exploration of quote offsets. In simple terms, SAC is trying to be profitable without becoming emotionally attached to one quoting style.

The soft state-action value function satisfies the soft Bellman equation:

$$Q^\pi(s_t,a_t) = r_t + \gamma\mathbb{E}_{s_{t+1}}\left[V^\pi(s_{t+1})\right], \tag{17}$$

with the soft value function defined by

$$V^\pi(s_t) = \mathbb{E}_{a_t\sim\pi}\left[Q^\pi(s_t,a_t) - \alpha\log\pi(a_t|s_t)\right]. \tag{18}$$

The critic is trained using targets of the form

$$y_t = r_t + \gamma\left(\min_{i=1,2} Q'_i(s_{t+1},a_{t+1}) - \alpha\log\pi(a_{t+1}|s_{t+1})\right), \tag{19}$$

where two critics are used to reduce overestimation bias. The actor is optimized to maximize expected Q-value while preserving entropy:

$$J_\pi = \mathbb{E}_{s_t\sim\mathcal{D},a_t\sim\pi}\left[\alpha\log\pi(a_t|s_t) - Q(s_t,a_t)\right]. \tag{20}$$

The temperature α can be fixed or automatically tuned.

SAC is especially important in this project because market making rewards are noisy, fills are stochastic, and toxicity can shift rapidly. In such conditions, entropy regularization often improves robustness and helps the policy avoid premature convergence. SAC therefore acts as a strong candidate for surpassing the Avellaneda-Stoikov benchmark when the environment contains jumps, regime changes, or nontrivial fill uncertainty. If PPO is the careful strategist, SAC is the strategist who also keeps a few extra plans in reserve because the world is, predictably, unreliable.

Algorithm 2 SAC Training

1: Initialize stochastic actor, two critics, and target critics
2: Sample actions using reparameterization
3: Store transitions in replay buffer
4: Update critics using the soft Bellman target
5: Update policy to maximize Q and entropy
6: Optionally tune the entropy temperature α
7: Soft-update target networks

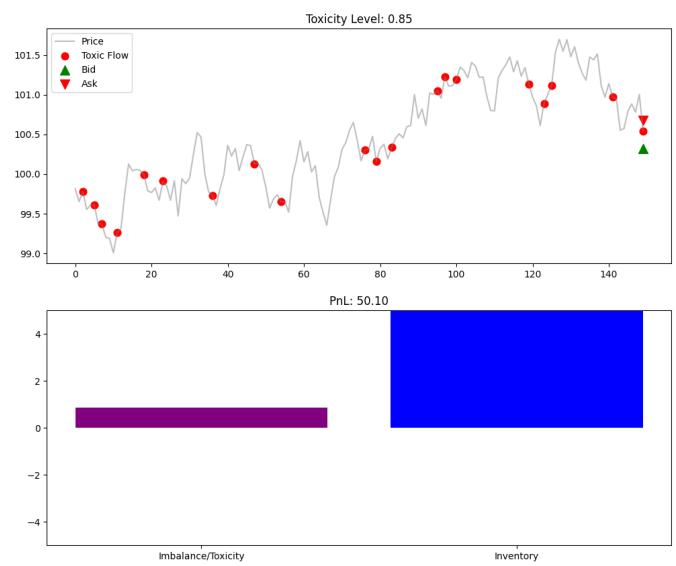


Fig. 2: SAC simulation output with entropy-driven exploration.

VII. Comparison Algorithms

The remaining algorithms are included primarily for comparison rather than as the central contribution. DQN and QR-DQN are value-based methods that work best after discretizing quote levels. A2C is an on-policy actor-critic baseline that is simple but can have higher variance. DDPG is an early deterministic continuous-control method, while TD3 improves it by reducing overestimation bias. TRPO is included because it provides a trust-region perspective that inspired PPO.

A. Deep Q-Network (DQN)

DQN estimates the action-value function $Q(s,a)$ and selects the action with the largest predicted value. The Bellman target

is

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-). \tag{21}$$

The loss is a squared TD error. In this project, DQN is only practical after action discretization, so it is useful as a baseline but not as the main method.

B. Quantile Regression DQN (QR-DQN)

QR-DQN extends DQN by learning quantiles of the return distribution. This is helpful for trading because return distributions are often heavy-tailed. The quantile regression loss gives more information than a single expected value and can therefore capture downside risk better than plain DQN.

C. Advantage Actor-Critic (A2C)

A2C learns both policy and value in parallel. It is a clean actor-critic baseline but its stochastic gradient updates can be noisy. It remains useful as a comparison point because it handles continuous decisions better than DQN without the extra engineering burden of more advanced methods.

D. Deep Deterministic Policy Gradient (DDPG)

DDPG is deterministic and continuous, but it is often sensitive to noise and approximation error. For market making this can be dangerous because quoting decisions should be stable under stochastic fills.

E. Twin Delayed DDPG (TD3)

TD3 improves DDPG by using two critics, delayed policy updates, and clipped target noise. It is more stable than DDPG and is a reasonable comparison baseline for continuous market-making control.

F. Trust Region Policy Optimization (TRPO)

TRPO constrains updates to a trust region in KL-divergence space. It is theoretically important and stable, but it is less convenient than PPO for the present project.

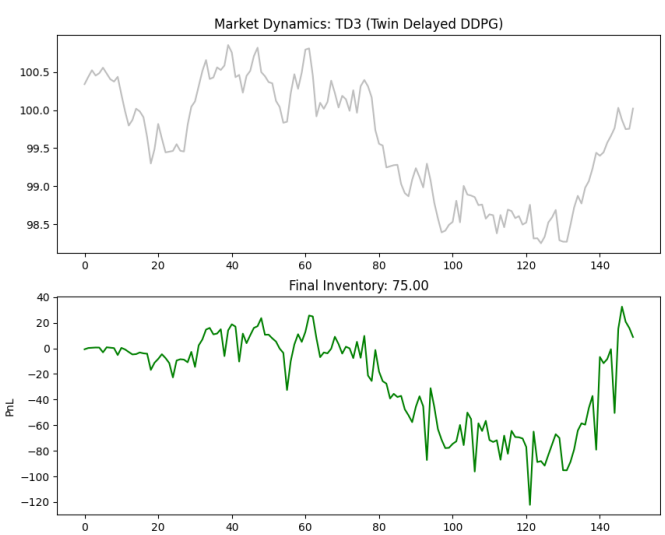


Fig. 3: TD3 simulation output used for comparison.

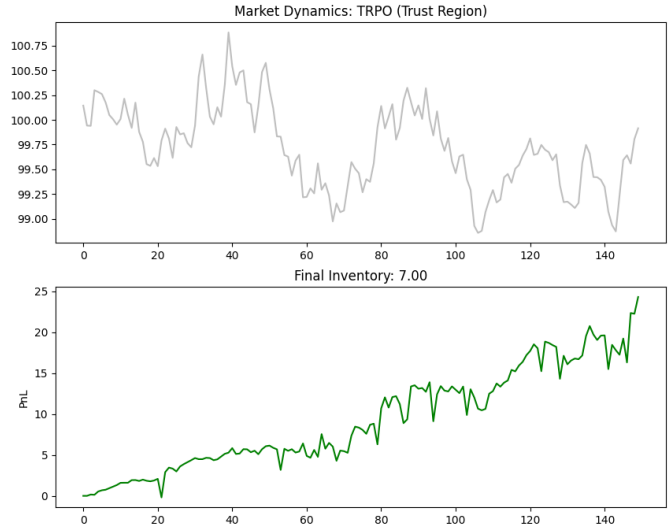
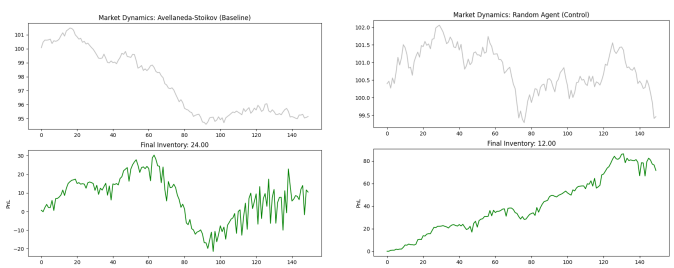


Fig. 4: TRPO simulation output used for comparison.

VIII. Simulation Results

The following plots summarize the benchmark and stress environments that were generated from the notebooks.



(a) Avellaneda-Stoikov baseline **(b)** Random-agent control

Fig. 5: Classical benchmark and control output.

The master comparison is the most important result because it directly addresses the project objective: whether an RL policy can outperform the analytical benchmark. In the generated simulations, PPO and SAC show the strongest behavior among the continuous-control methods, which is consistent with their design. The comparison should be interpreted carefully, however, because outperformance depends on the training environment, reward shaping, and assumptions built into execution. Still, the result is meaningful because it demonstrates that a learned policy can adapt to dynamics that the classical solution does not model explicitly.

TABLE I: Comparison of Methods

Method	Role	Strength	Limitation
PPO	Main method	Stable continuous control	On-policy sample cost
SAC	Main method	Exploration and robustness	More components
DQN	Comparison	Simple value baseline	Discrete actions only
QR-DQN	Comparison	Distributional awareness	Still discretized
A2C	Comparison	Simple actor-critic	Higher variance
DDPG	Comparison	Deterministic control	Sensitive to noise
TD3	Comparison	Reduced overestimation	More tuning
TRPO	Comparison	Trust-region stability	Computational cost

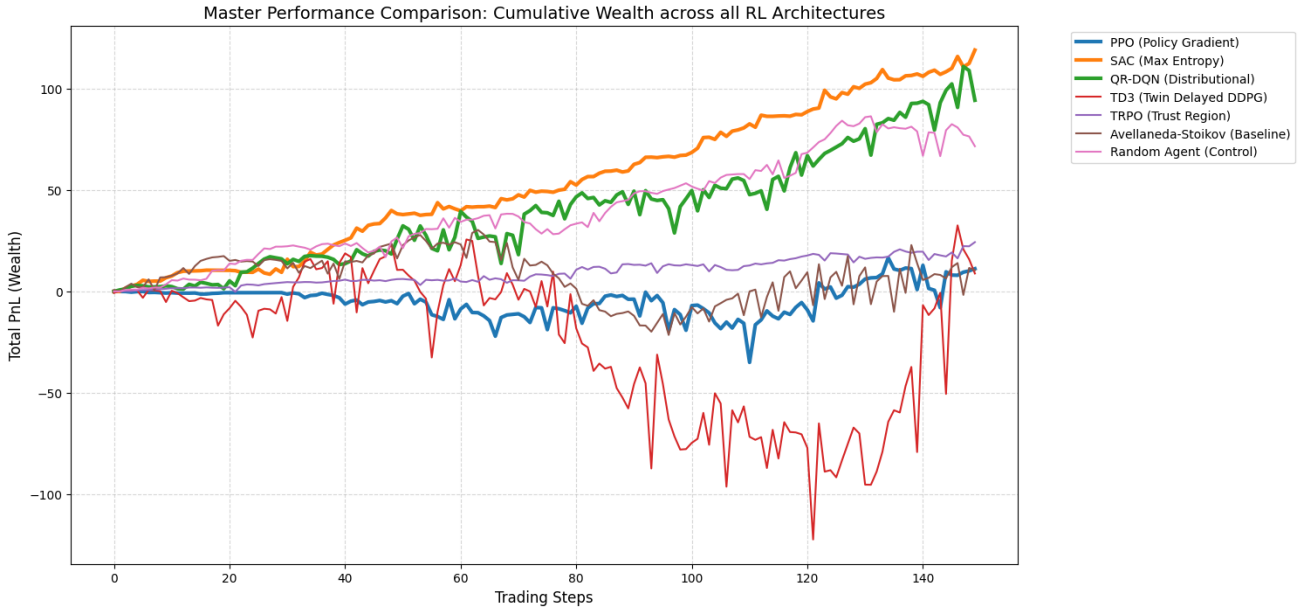


Fig. 6: Master comparison of cumulative wealth across all RL architectures and baselines.

IX. Conclusion

This paper studied market making as a reinforcement learning problem with the explicit goal of surpassing the Avellaneda-Stoikov analytical benchmark. The main contribution is the comparative analysis of two continuous-control methods, PPO and SAC, which are better aligned with the structure of the quoting problem than discrete value-based methods. The derivations show why the problem naturally favors continuous action policies, and the simulations illustrate how the learned agents behave under normal, toxic, and stress environments.

PPO is a strong choice because it balances stability and expressiveness through clipped policy updates. SAC is also highly suitable because entropy regularization improves robustness in noisy and nonstationary settings. The comparison algorithms are useful for validating the setup, but the project's central message is that continuous RL methods are the right tools for learning quoting policies that may exceed the classical benchmark when the environment is sufficiently rich.

Acknowledgment

The author thanks the course instructors and project supervisors for guidance and feedback.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 2015.
[2] J. Schulman et al., "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.
[3] T. Haarnoja et al., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," ICML 2018.
[4] S. Fujimoto, H. Van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," ICML 2018.
[5] T. Lillicrap et al., "Continuous control with deep reinforcement learning," arXiv:1509.02971, 2015.
[6] J. Schulman et al., "Trust Region Policy Optimization," ICML 2015.
[7] W. Dabney et al., "Distributional Reinforcement Learning with Quantile Regression," AAAI 2018.
[8] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," Quantitative Finance, 2008.